

Student Name: Frederick Nguyen
Student Id: 016467373

Student Name: Ali Halmamat
Student Id: 017505410

A. Coding

Shell1.c

```
/**
 * Description:
 * -----
 * This program implements a minimal shell for Assignment 3.
 *
 * When the user types:
 *
 *     ./countnames file1.txt file2.txt file3.txt
 *
 * The shell will:
 *
 *   - Fork once per input file
 *   - Each child executes ./countnames on ONE file
 *   - Redirect stdout to PID.out
 *   - Redirect stderr to PID.err
 *   - Parent waits for all children (parallel execution)
 *
 * This removes all APUE dependencies and uses standard POSIX calls.
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_LINE 1024
#define MAX_ARGS 256
#define MAX_NAMES 300

typedef struct {
    char name[30];
    int count;
} NameCountData;
```

```

typedef enum {
    TYPE_NAMECOUNT,
    TYPE_B
} MessageType;

typedef struct {
    MessageType type;
    size_t size;
} MessageHeader;

typedef struct {
    pid_t pid;
    int read_fd;
} ChildPipe;

static int split_line(char *line, char *argv[], int max_args) {
    int argc = 0;
    char *tok = strtok(line, " \t");
    while (tok != NULL && argc < max_args - 1) {
        argv[argc++] = tok;
        tok = strtok(NULL, " \t");
    }
    argv[argc] = NULL;
    return argc;
}

static int is_countnames_cmd(const char *cmd) {
    return strcmp(cmd, "./countnames") == 0 || strcmp(cmd, "countnames") == 0;
}

static void add_or_sum(NameCountData totals[], int *distinct, const NameCountData
*item) {
    for (int i = 0; i < *distinct; i++) {
        if (strcmp(totals[i].name, item->name) == 0) {
            totals[i].count += item->count;
            return;
        }
    }

    if (*distinct < MAX_NAMES) {
        strncpy(totals[*distinct].name, item->name, sizeof(totals[*distinct].name) -
1);
        totals[*distinct].name[sizeof(totals[*distinct].name) - 1] = '\0';
        totals[*distinct].count = item->count;
    }
}

```

```

        (*distinct)++;
    }
}

static ssize_t read_all(int fd, void *buf, size_t count) {
    size_t total = 0;
    char *p = (char *)buf;

    while (total < count) {
        ssize_t n = read(fd, p + total, count - total);
        if (n == 0) {
            return (ssize_t)total;    /* EOF */
        }
        if (n < 0) {
            return -1;
        }
        total += (size_t)n;
    }

    return (ssize_t)total;
}

static void read_from_pipe(int fd, NameCountData totals[], int *distinct) {
    MessageHeader header;

    while (1) {
        ssize_t n = read_all(fd, &header, sizeof(header));
        if (n == 0) {
            return;    /* end of pipe */
        }
        if (n < 0 || n != (ssize_t)sizeof(header)) {
            fprintf(stderr, "Error reading message header from pipe\n");
            return;
        }

        switch (header.type) {
            case TYPE_NAMECOUNT: {
                NameCountData dataNC;

                if (header.size != sizeof(NameCountData)) {
                    fprintf(stderr, "Invalid payload size for TYPE_NAMECOUNT\n");
                    return;
                }

                n = read_all(fd, &dataNC, header.size);
            }
        }
    }
}

```

```

        if (n < 0 || n != (ssize_t)header.size) {
            fprintf(stderr, "Error reading NameCountData payload\n");
            return;
        }

        add_or_sum(totals, distinct, &dataNC);
        break;
    }

    case TYPE_B: {
        /* future extension */
        char discard[256];
        size_t remaining = header.size;

        while (remaining > 0) {
            size_t chunk = remaining < sizeof(discard) ? remaining :
sizeof(discard);
            ssize_t got = read_all(fd, discard, chunk);
            if (got <= 0) {
                fprintf(stderr, "Error reading TYPE_B payload\n");
                return;
            }
            remaining -= (size_t)got;
        }
        break;
    }

    default:
        fprintf(stderr, "Unknown message type received: %d\n", header.type);
        return;
}
}
}

int main(void) {
    char line[MAX_LINE];

    while (1) {
        printf("%s ");
        fflush(stdout);

        if (fgets(line, sizeof(line), stdin) == NULL) {
            break;
        }
    }
}

```

```

line[strcspn(line, "\n")] = '\0';

if (line[0] == '\0') {
    continue;
}

if (strcmp(line, "exit") == 0 || strcmp(line, "quit") == 0) {
    break;
}

char line_copy[MAX_LINE];
char *args[MAX_ARGS];

strncpy(line_copy, line, sizeof(line_copy) - 1);
line_copy[sizeof(line_copy) - 1] = '\0';

int argc = split_line(line_copy, args, MAX_ARGS);
if (argc == 0) {
    continue;
}

if (!is_countnames_cmd(args[0])) {
    pid_t other = fork();
    if (other == 0) {
        execvp(args[0], args);
        _exit(1);
    } else if (other > 0) {
        waitpid(other, NULL, 0);
    }
    continue;
}

if (argc == 1) {
    pid_t pid = fork();
    if (pid == 0) {
        char *cn_argv[] = { (char *)"./countnames", NULL };
        execvp(cn_argv[0], cn_argv);
        _exit(1);
    } else if (pid > 0) {
        waitpid(pid, NULL, 0);
    }
    continue;
}

ChildPipe children[MAX_ARGS];

```

```

int child_count = 0;
NameCountData totals[MAX_NAMES];
int total_distinct = 0;

for (int i = 0; i < MAX_NAMES; i++) {
    totals[i].name[0] = '\0';
    totals[i].count = 0;
}

/*
 * PARALLEL PART:
 * create all children first, do not wait here.
 */
for (int i = 1; i < argc; i++) {
    int pipefd[2];

    if (pipe(pipefd) < 0) {
        perror("pipe");
        continue;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        close(pipefd[0]);
        close(pipefd[1]);
        continue;
    }

    if (pid == 0) {
        close(pipefd[0]);

        /*
         * Replace stdout with write end of pipe.
         * Child can now "print" / write to stdout and parent receives it.
         */
        if (dup2(pipefd[1], STDOUT_FILENO) < 0) {
            close(pipefd[1]);
            _exit(1);
        }

        close(pipefd[1]);

        char *cn_argv[] = { (char *)"./countnames", args[i], NULL };
    }
}

```

```

        execvp(cn_argv[0], cn_argv);
        _exit(1);
    }

    close(pipefd[1]);
    children[child_count].pid = pid;
    children[child_count].read_fd = pipefd[0];
    child_count++;
}

/*
 * Wait for children after all are spawned.
 * wait() returns PID of child that finished.
 */
int status;
pid_t pid;

while ((pid = wait(&status)) > 0) {
    if (WIFEXITED(status)) {
        fprintf(stderr,
            "Child %d terminated normally with exit code: %d\n",
            pid, WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        fprintf(stderr,
            "Child %d terminated abnormally with signal number: %d\n",
            pid, WTERMSIG(status));
    }

    for (int i = 0; i < child_count; i++) {
        if (children[i].pid == pid) {
            read_from_pipe(children[i].read_fd, totals, &total_distinct);
            close(children[i].read_fd);
            children[i].read_fd = -1;
            break;
        }
    }
}

/* print final aggregated result */
for (int i = 0; i < total_distinct; i++) {
    printf("%s: %d\n", totals[i].name, totals[i].count);
}

return 0;

```

```
}
```

Countnames.c

```
/**
 * Description:
 * -----
 * countnames reads names from a file (or stdin) and counts
 * how many times each name appears.
 *
 * Author names: Frederick Nguyen | Ali Halmamat
 * Author emails: quangnguyen1705@gmail.com | ali.Halmamat@sjsu.edu
 * Last modified date: 03/21/2026
 * Creation date: 03/21/2026
 * For Assignment 3:
 * - This program processes ONE file at a time.
 * - Output is written to stdout.
 * - Warnings and errors are written to stderr.
 * - The parent shell process redirects stdout/stderr
 *   to PID.out and PID.err.
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_NAMES 100
#define MAX_NAME_LEN 30
#define BUF_SIZE 256

typedef struct {
    char name[30];
    int count;
} NameCountData;

typedef enum {
    TYPE_NAMECOUNT,
    TYPE_B
} MessageType;
```



```

typedef struct {
    MessageType type;
    size_t size;
} MessageHeader;

static void strip_newline(char *s) {
    size_t len = strlen(s);
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r')) {
        s[len - 1] = '\0';
        len--;
    }
}

static ssize_t write_all(int fd, const void *buf, size_t count) {
    size_t total = 0;
    const char *p = (const char *)buf;

    while (total < count) {
        ssize_t n = write(fd, p + total, count - total);
        if (n <= 0) {
            return -1;
        }
        total += (size_t)n;
    }

    return (ssize_t)total;
}

static int write_struct_namecount(int fd, const NameCountData *data) {
    MessageHeader header;
    header.type = TYPE_NAMECOUNT;
    header.size = sizeof(NameCountData);

    if (write_all(fd, &header, sizeof(header)) != (ssize_t)sizeof(header)) {
        return -1;
    }

    if (write_all(fd, data, sizeof(NameCountData)) !=
        (ssize_t)sizeof(NameCountData)) {
        return -1;
    }

    return 0;
}

```

```

int main(int argc, char *argv[]) {
    FILE *in = NULL;
    FILE *outf = NULL;
    FILE *errf = NULL;
    char outname[64];
    char errname[64];
    pid_t pid = getpid();

    NameCountData table[MAX_NAMES];
    int distinct = 0;
    char buf[BUF_SIZE];
    int line_num = 0;
    const char *src_label = "stdin";

    for (int i = 0; i < MAX_NAMES; i++) {
        table[i].name[0] = '\0';
        table[i].count = 0;
    }

    snprintf(outname, sizeof(outname), "%d.out", (int)pid);
    snprintf(errname, sizeof(errname), "%d.err", (int)pid);

    outf = fopen(outname, "w");
    if (outf == NULL) {
        return 1;
    }

    errf = fopen(errname, "w");
    if (errf == NULL) {
        fclose(outf);
        return 1;
    }

    if (argc == 2) {
        src_label = argv[1];
        in = fopen(argv[1], "r");
        if (in == NULL) {
            fprintf(errf, "error: cannot open file %s\n", argv[1]);
            fclose(outf);
            fclose(errf);
            return 1;
        }
    } else if (argc == 1) {
        in = stdin;
    }
}

```

```

    } else {
        fclose(outf);
        fclose(errf);
        return 0;
    }

    while (fgets(buf, sizeof(buf), in) != NULL) {
        line_num++;
        strip_newline(buf);

        if (strlen(buf) < 1) {
            fprintf(errf, "Warning - file %s line %d is empty.\n", src_label,
line_num);
            continue;
        }

        if (strlen(buf) > MAX_NAME_LEN) {
            buf[MAX_NAME_LEN] = '\0';
        }

        int found = 0;
        for (int i = 0; i < distinct; i++) {
            if (strcmp(table[i].name, buf) == 0) {
                table[i].count++;
                found = 1;
                break;
            }
        }

        if (!found) {
            if (distinct >= MAX_NAMES) {
                fprintf(errf, "error: too many distinct names\n");
                if (in != stdin) {
                    fclose(in);
                }
                fclose(outf);
                fclose(errf);
                return 1;
            }

            strncpy(table[distinct].name, buf, sizeof(table[distinct].name) - 1);
            table[distinct].name[sizeof(table[distinct].name) - 1] = '\0';
            table[distinct].count = 1;
            distinct++;
        }
    }

```

```

}

if (in != stdin) {
    fclose(in);
}

/* write per-child result to PID.out */
for (int i = 0; i < distinct; i++) {
    fprintf(outf, "%s: %d\n", table[i].name, table[i].count);
}

fclose(outf);
fclose(errf);

/*
 * stdout of child has been redirected by shell.c into a pipe.
 * So sending to STDOUT_FILENO sends data back to the parent.
 */
for (int i = 0; i < distinct; i++) {
    if (write_struct_namecount(STDOUT_FILENO, &table[i]) < 0) {
        return 1;
    }
}

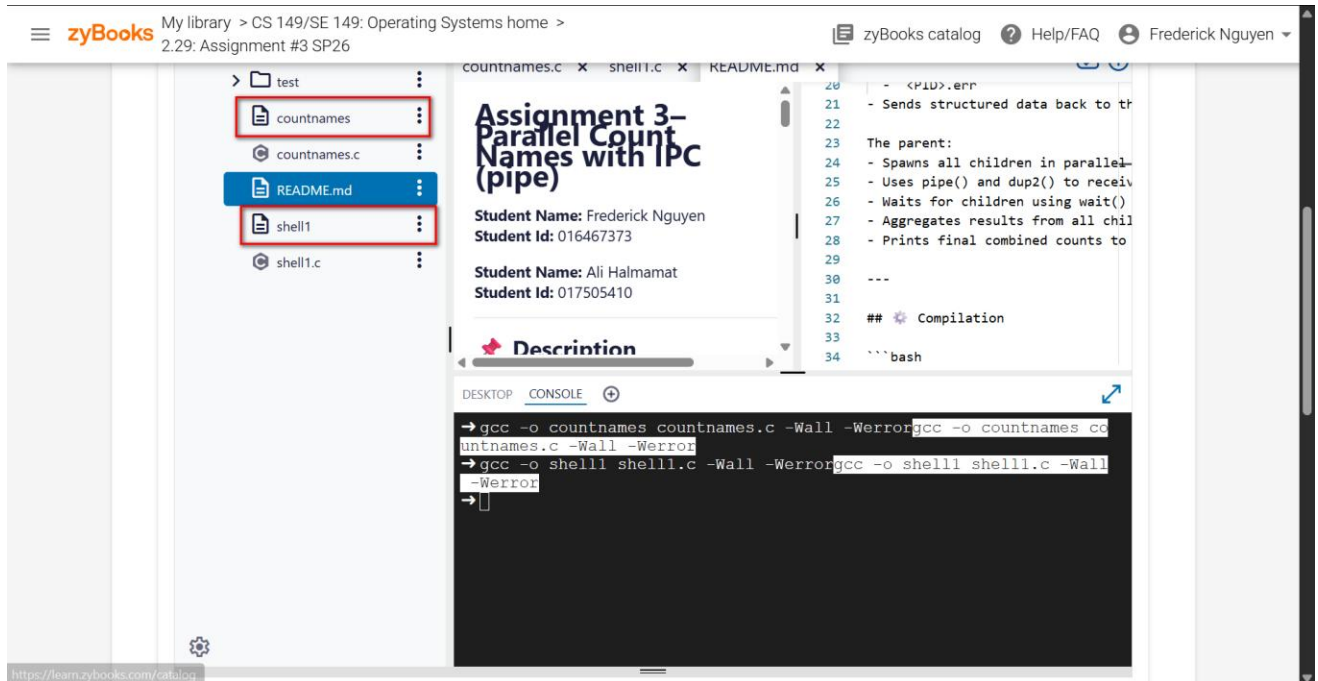
return 0;
}

```

B. Result and TestCase

Run compilation: `gcc -o countnames countnames.c -Wall -Werror`

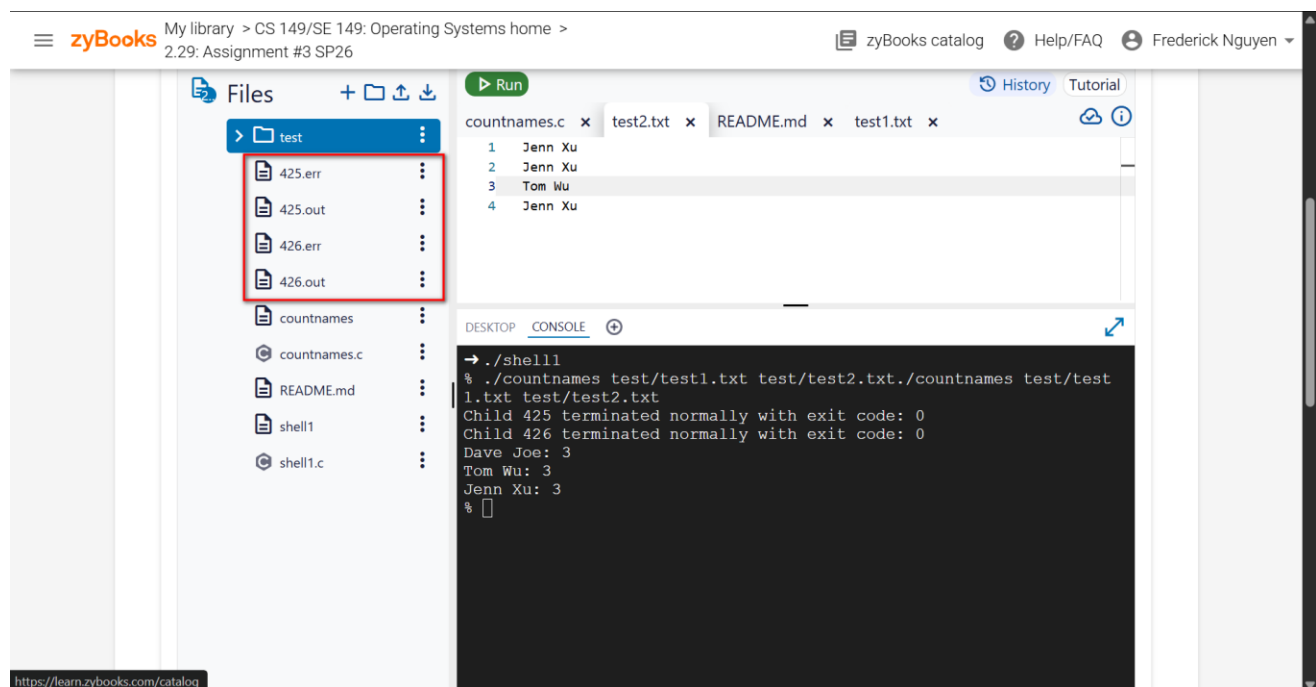
`gcc -o shell1 shell1.c -Wall -Werror`



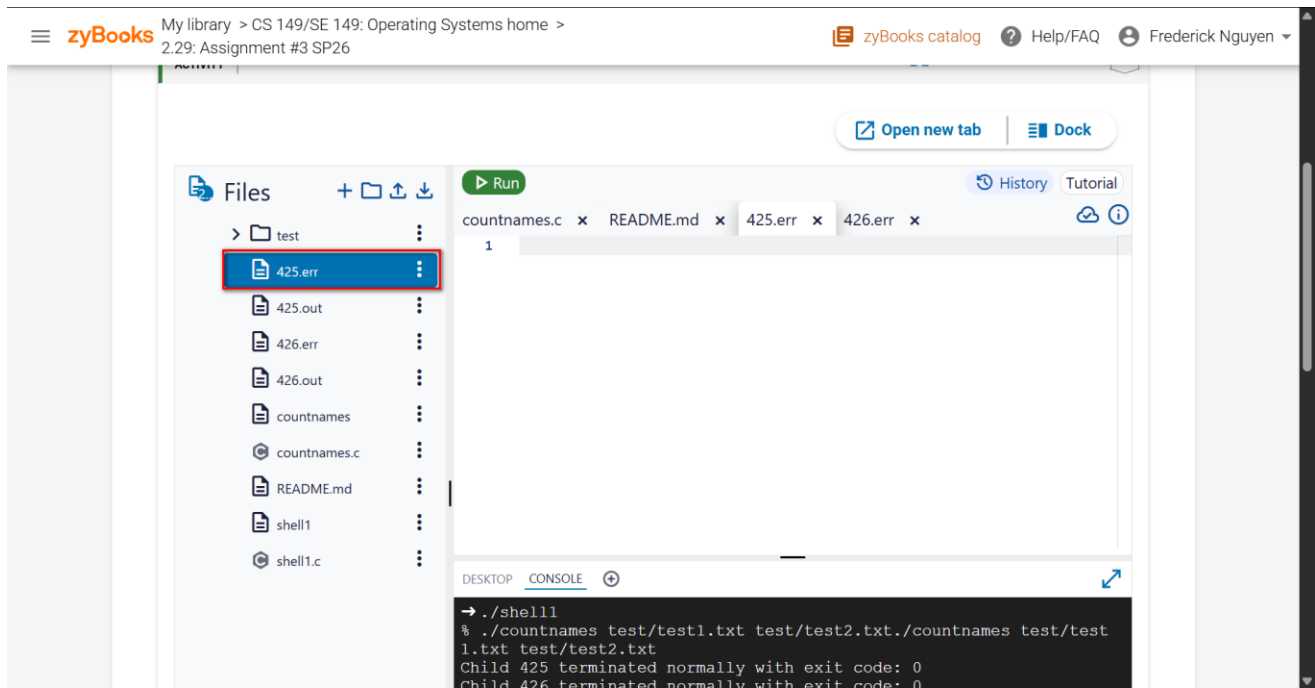
1. Test Case 1 – Basic aggregation

Run Command: `./shell1`

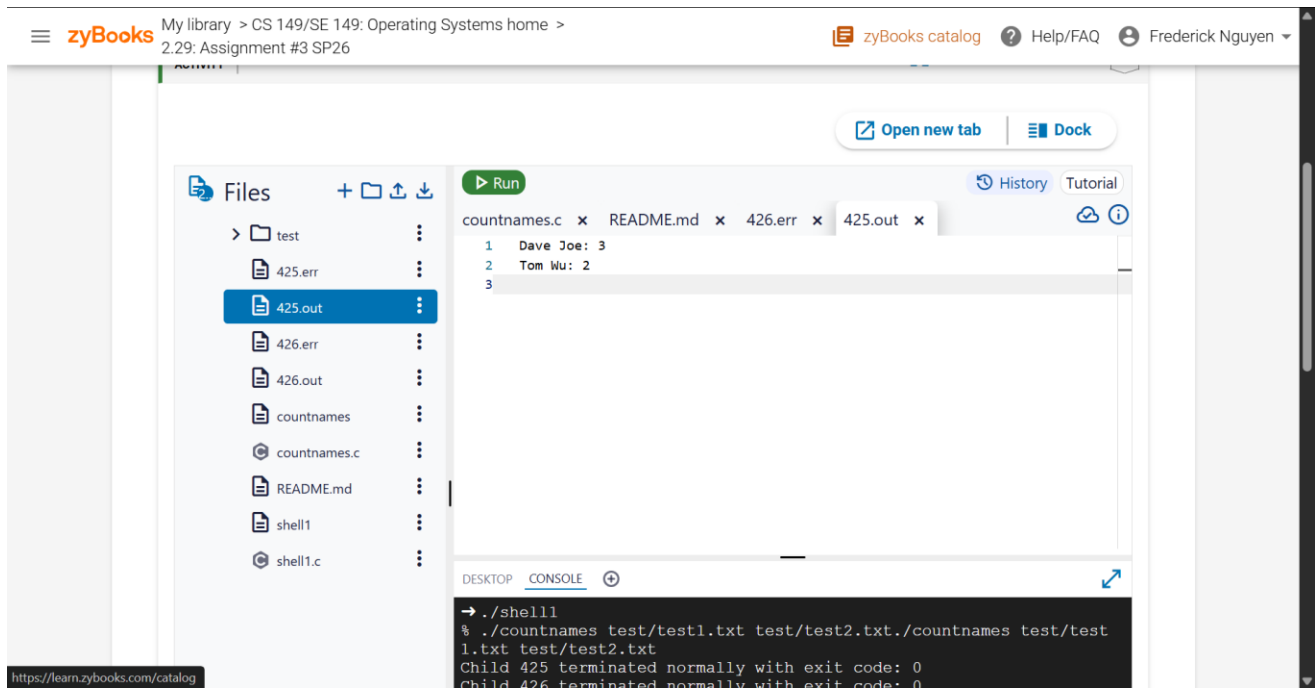
`./countnames test/test1.txt test/test2.txt`



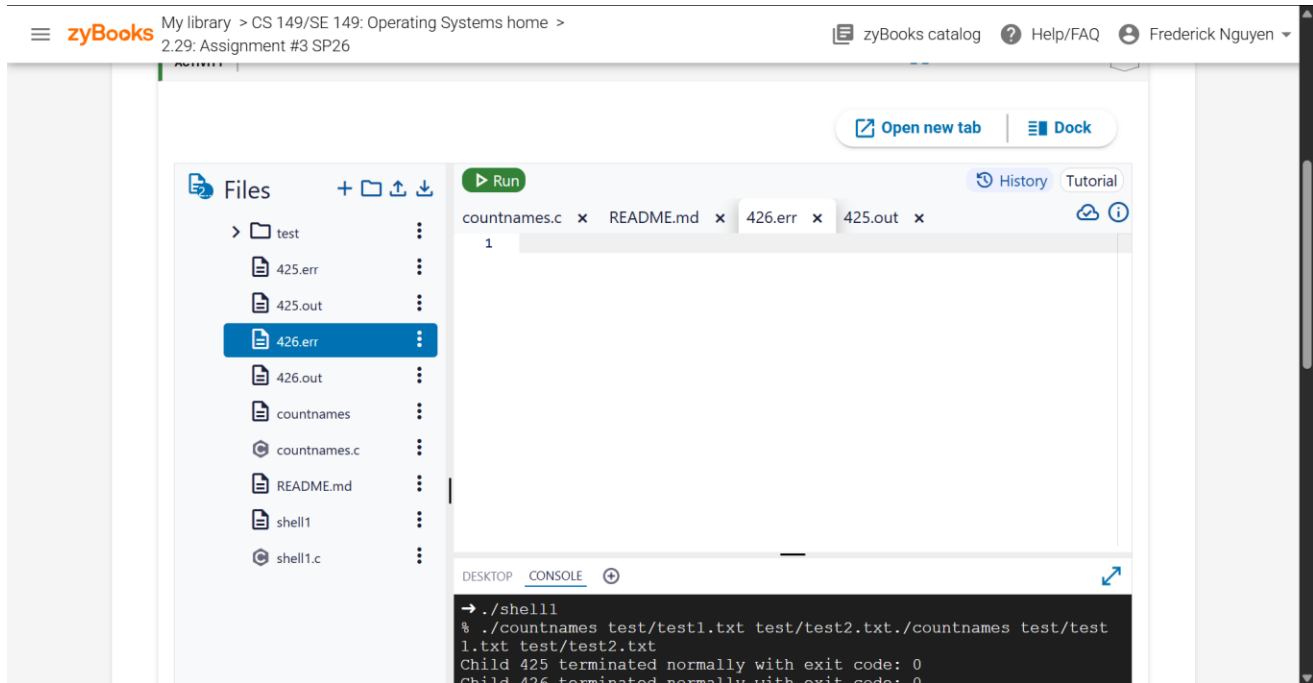
425.err



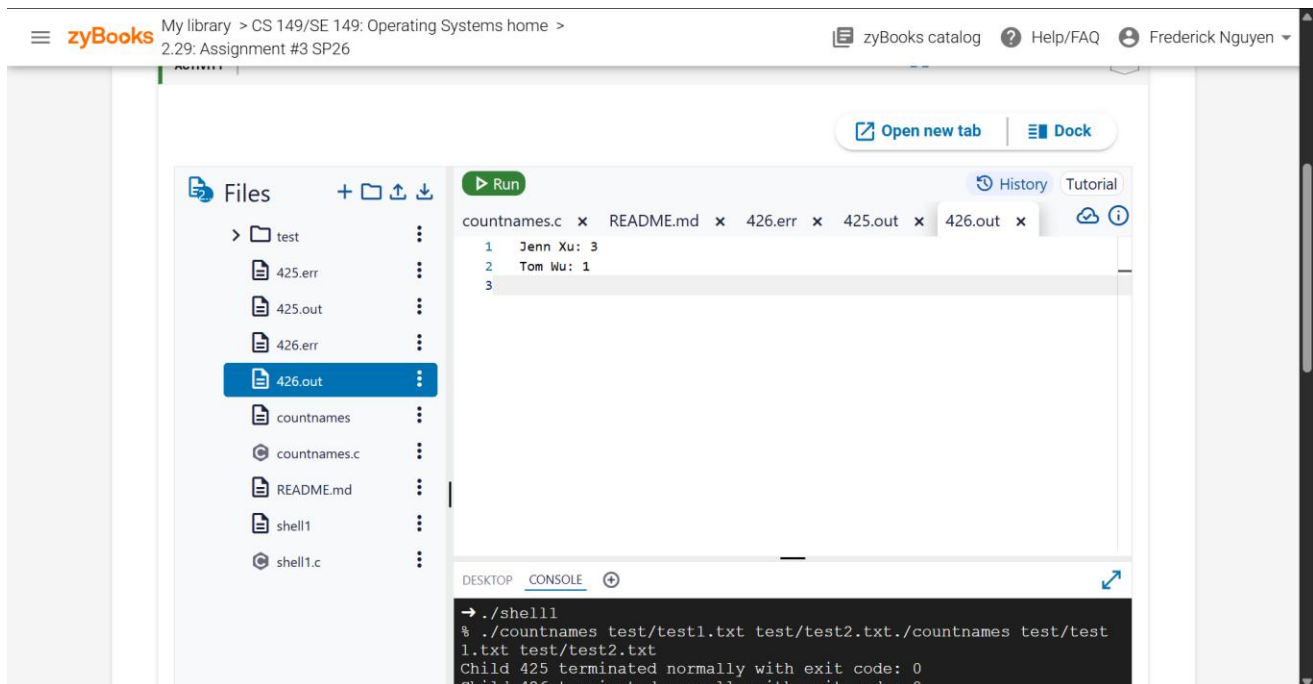
425.out



426.err



426.out



2. Test Case 2 – Test Case 2 – Missing file.

Run command: `./shell1`

`./countnames test/missing.txt test/names1.txt`

zyBooks My library > CS 149/SE 149: Operating Systems home > 2.29: Assignment #3 SP26 zyBooks catalog Help/FAQ Frederick Nguyen

test1.txt
test2.txt
425.err
425.out
426.err
426.out
441.err
441.out
442.err
442.out
countnames
countnames.c
README.md
shell1
shell1.c

72
73
74 ### **Test Case 2: Missing file**
75
76 Command:
77 bash

DESKTOP CONSOLE

```
→ ./shell1  
% ./countnames test/missing.txt test/names1.txt  
Child 441 terminated normally with exit code: 1  
Child 442 terminated normally with exit code: 0  
Tom Wu: 3  
% 
```

<https://learn.zybooks.com/catalog/> **submit for grading**

441.out

zyBooks My library > CS 149/SE 149: Operating Systems home > 2.29: Assignment #3 SP26 zyBooks catalog Help/FAQ Frederick Nguyen

Files + Run History Tutorial

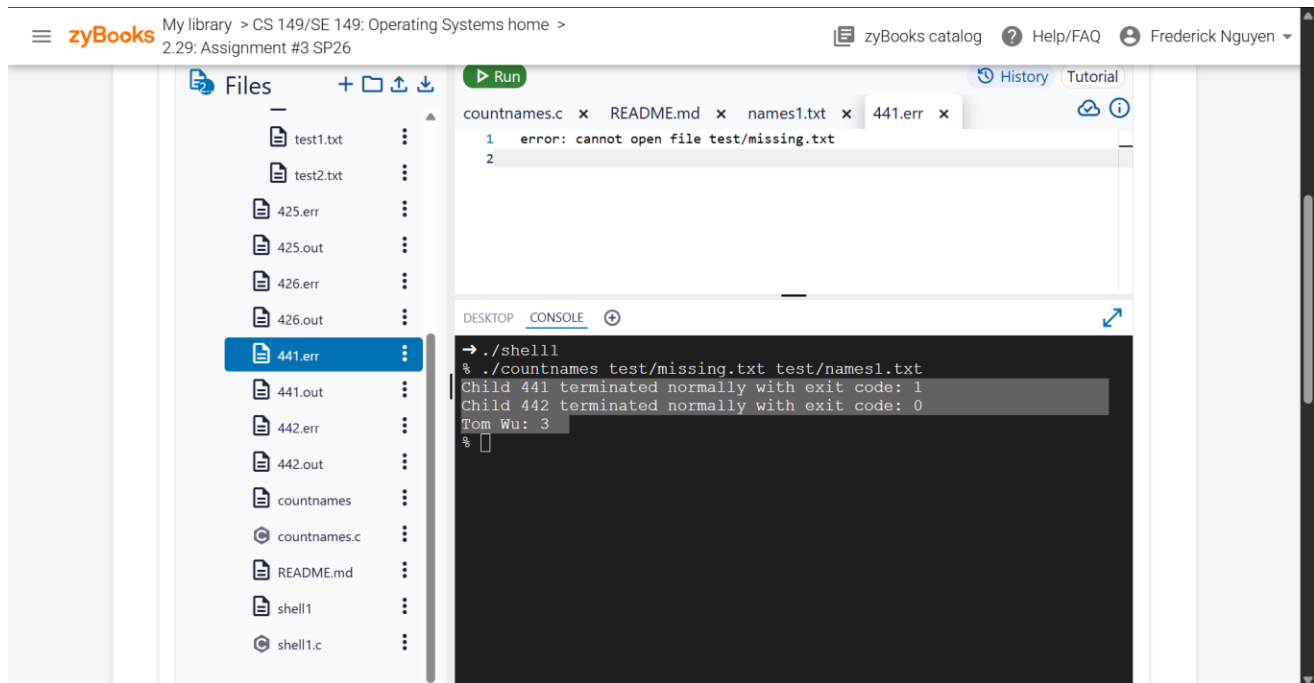
countnames.c x README.md x names1.txt x 441.out x

1

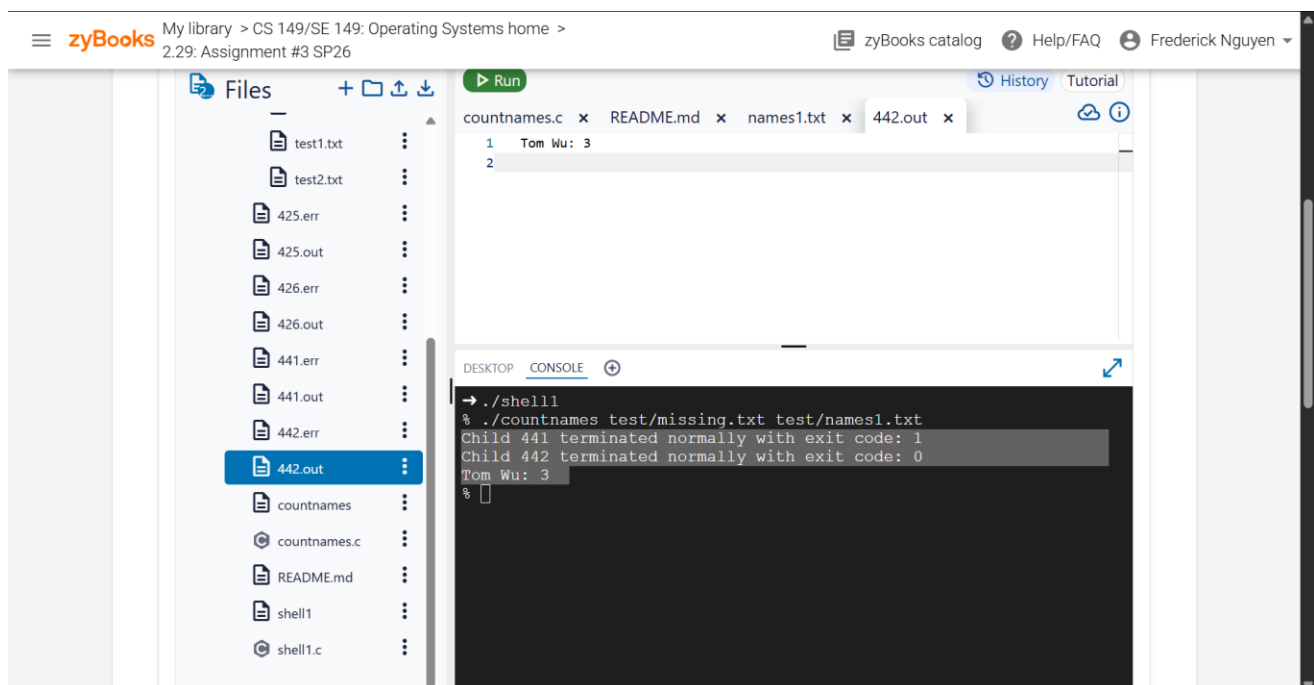
DESKTOP CONSOLE

```
→ ./shell1  
% ./countnames test/missing.txt test/names1.txt  
Child 441 terminated normally with exit code: 1  
Child 442 terminated normally with exit code: 0  
Tom Wu: 3  
% 
```

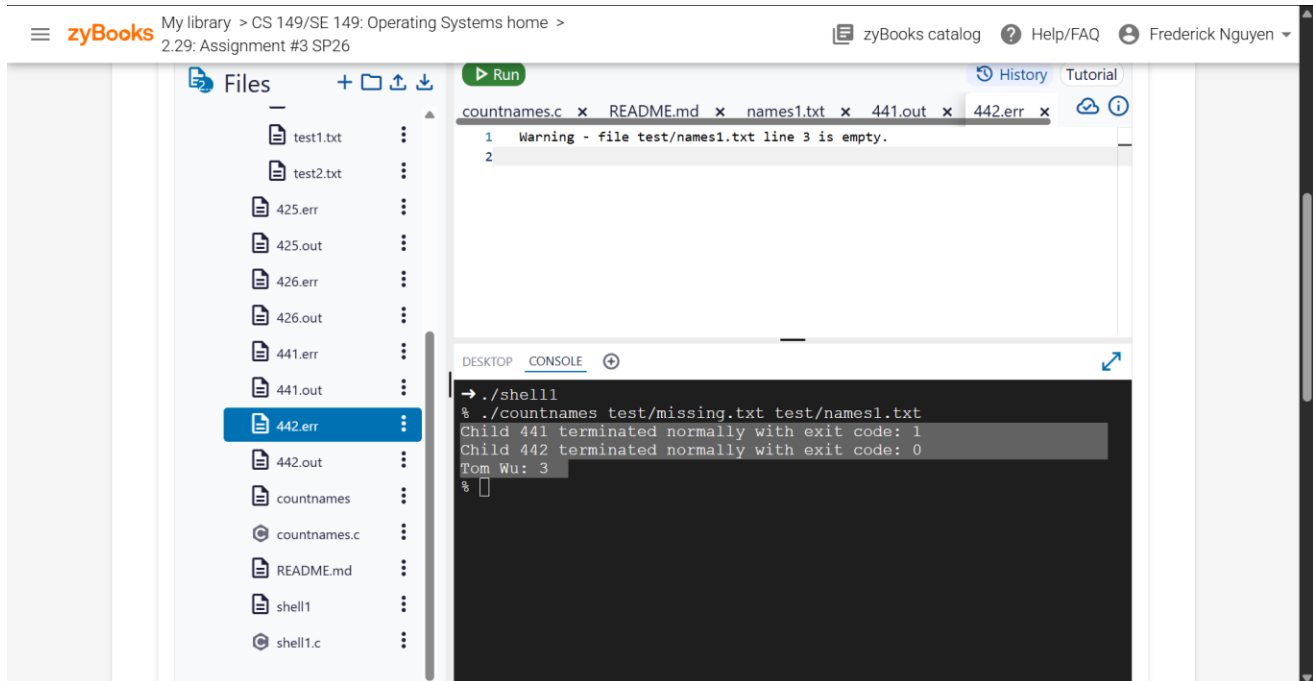
441.err



442.out



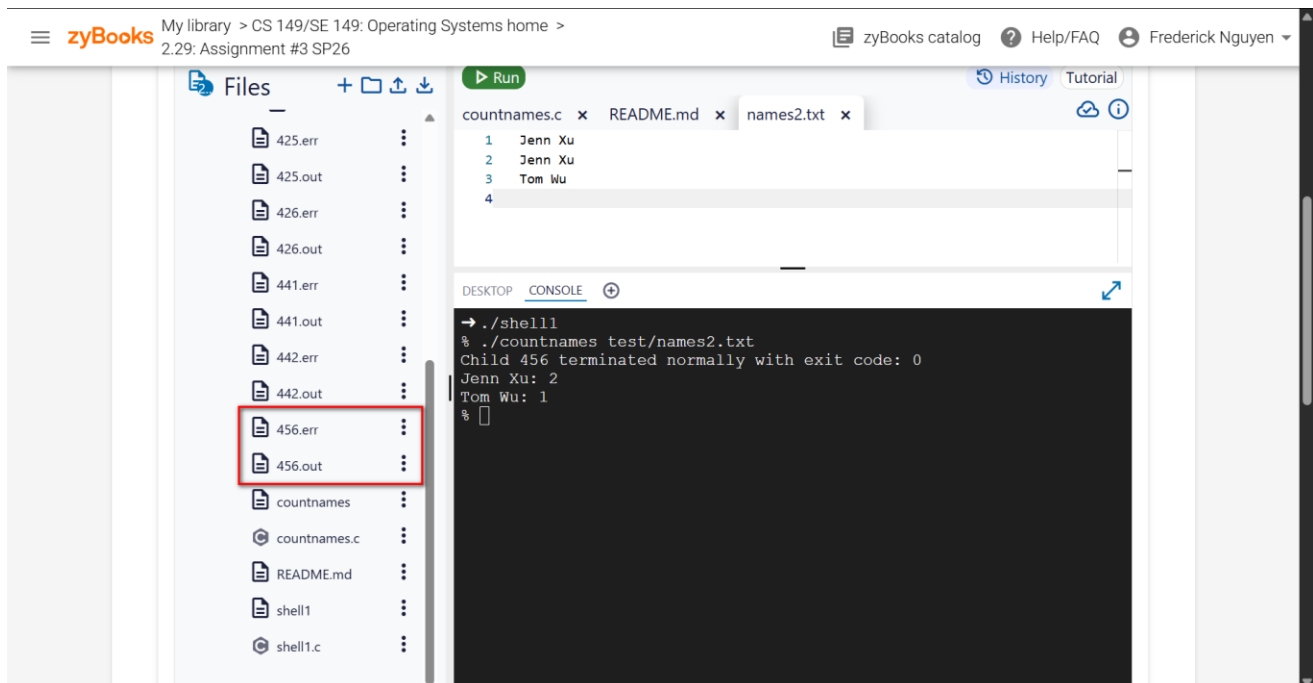
442.err



3. Test Case 3 – Single file

Run Command: `./shell1`

`./countnames test/name2.txt`



456.err

zyBooks My library > CS 149/SE 149: Operating Systems home > 2.29: Assignment #3 SP26 zyBooks catalog Help/FAQ Frederick Nguyen

Files + Run History Tutorial

countnames.c x README.md x names2.txt x 456.err x

1

DESKTOP CONSOLE

```
→ ./shell1
% ./countnames test/names2.txt
Child 456 terminated normally with exit code: 0
Jenn Xu: 2
Tom Wu: 1
% 
```

https://learn.zybooks.com/catalog

456.out

zyBooks My library > CS 149/SE 149: Operating Systems home > 2.29: Assignment #3 SP26 zyBooks catalog Help/FAQ Frederick Nguyen

Files + Run History Tutorial

countnames.c x README.md x names2.txt x 456.out x

1 Jenn Xu: 2
2 Tom Wu: 1
3

DESKTOP CONSOLE

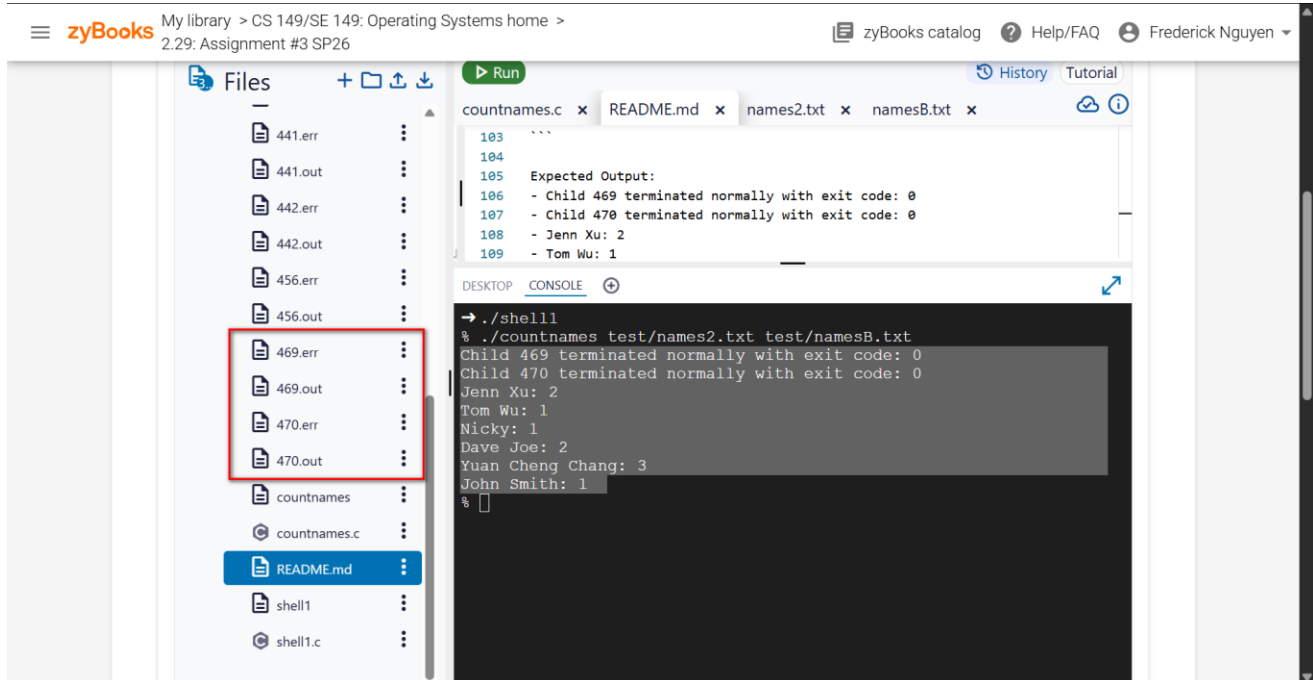
```
→ ./shell1
% ./countnames test/names2.txt
Child 456 terminated normally with exit code: 0
Jenn Xu: 2
Tom Wu: 1
% 
```

https://learn.zybooks.com/catalog

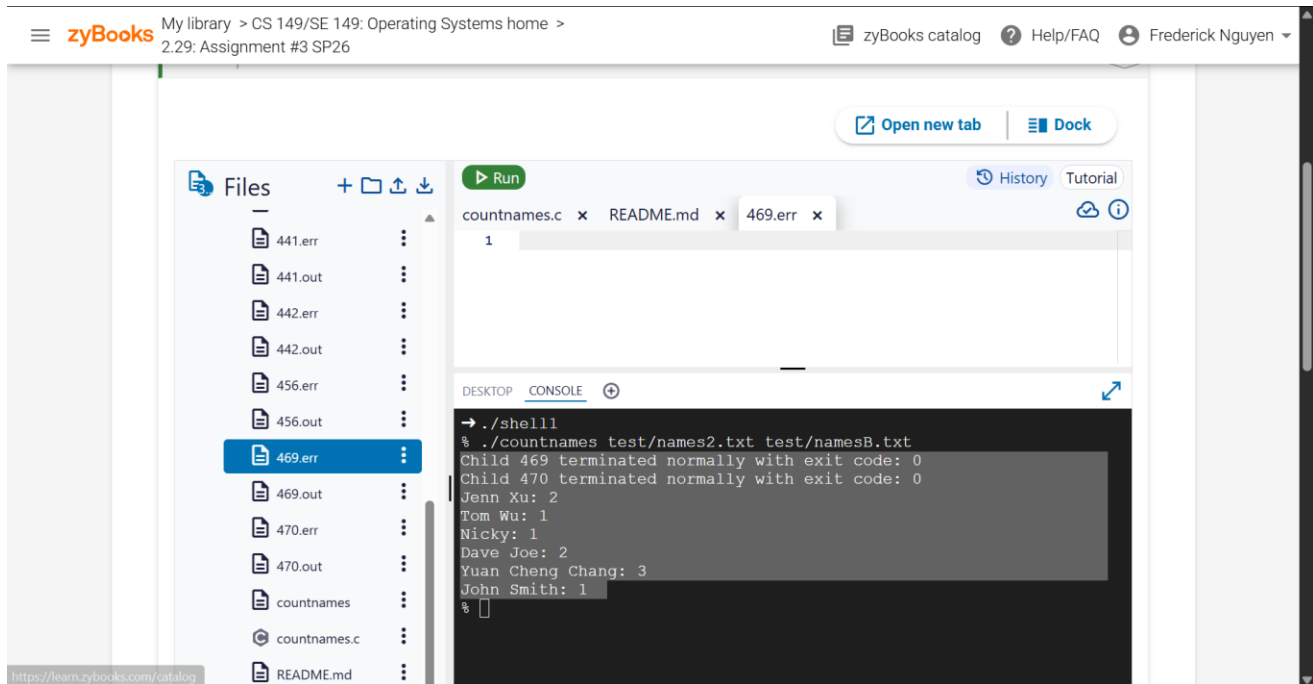
4. Test Case 4 – Empty line handling

Run Command: `./shell1`

`./countnames test/name2.txt`



469.err



469.out

zyBooks My library > CS 149/SE 149: Operating Systems home > 2.29: Assignment #3 SP26

zyBooks catalog ? Help/FAQ Frederick Nguyen

Open new tab Dock

Files

- 441.err
- 441.out
- 442.err
- 442.out
- 456.err
- 456.out
- 469.err
- 469.out**
- 470.err
- 470.out
- countnames
- countnames.c
- README.md

Run

countnames.c x README.md x 469.out x

```
1 Jenn Xu: 2
2 Tom Wu: 1
3
```

DESKTOP CONSOLE

```
→ ./shell1
% ./countnames test/names2.txt test/namesB.txt
Child 469 terminated normally with exit code: 0
Child 470 terminated normally with exit code: 0
Jenn Xu: 2
Tom Wu: 1
Nicky: 1
Dave Joe: 2
Yuan Cheng Chang: 3
John Smith: 1
%
```

470.err

zyBooks My library > CS 149/SE 149: Operating Systems home > 2.29: Assignment #3 SP26

zyBooks catalog ? Help/FAQ Frederick Nguyen

Group 46: Ali Halmamat, Frederick Nguyen

LAB ACTIVITY 2.29.1: Code Full screen 0 / 100

Open new tab Dock

Files

- 441.err
- 441.out
- 442.err
- 442.out
- 456.err
- 456.out
- 469.err
- 469.out
- 470.err**
- 470.out

Run

countnames.c x README.md x 470.err x

```
1 Warning - file test/namesB.txt line 2 is empty.
2 Warning - file test/namesB.txt line 5 is empty.
3
```

DESKTOP CONSOLE

```
→ ./shell1
% ./countnames test/names2.txt test/namesB.txt
Child 469 terminated normally with exit code: 0
Child 470 terminated normally with exit code: 0
Jenn Xu: 2
Tom Wu: 1
Nicky: 1
Dave Joe: 2
Yuan Cheng Chang: 3
```

470.out

zyBooks

My library > CS 149/SE 149: Operating Systems home > 2.29: Assignment #3 SP26

zyBooks catalog ? Help/FAQ Frederick Nguyen

Group 46: Ali Halmamat, Frederick Nguyen

LAB ACTIVITY

2.29.1: Code

Full screen 0 / 100

Open new tab Dock

Files

441.err

441.out

442.err

442.out

456.err

456.out

469.err

469.out

470.err

470.out

Run

countnames.c x README.md x 470.out x

```
1 Nicky: 1
2 Dave Joe: 2
3 Yuan Cheng Chang: 3
4 John Smith: 1
5
```

DESKTOP CONSOLE

```
→ ./shell1
% ./countnames test/names2.txt test/namesB.txt
Child 469 terminated normally with exit code: 0
Child 470 terminated normally with exit code: 0
Jenn Xu: 2
Tom Wu: 1
Nicky: 1
Dave Joe: 2
Yuan Cheng Chang: 3
```